

逐梦实验室

BACKEND

Hello, World!

写给还没写过一行代码的你：
从这一行「Hello, World!」开始，认识后端

今天，我们只讲四件事

01 后端到底是什么？

用你熟悉的《我的世界》和「淘宝下单」，把 **前端** → **后端** → **数据库** 这条链路完整走一遍，看清后端站在哪个位置、凭什么不可替代

03 接下来该怎么学？

一条能真正走通的路线：用基础知识打好地基，同时尝试用 **Java** 或 **Go** 做出东西来，最后按自己的方向深入

02 第一行代码长什么样？

用 **Java** 和 **Go** 写出第一个 HelloWorld

04 实验室考核

不要求有多好的基础，主要考察你的学习是否能坚持下来，是否有一定的学习能力

今天你只需要知道后端在做什么，以及你的第一步要怎么开始

先从一个你一定玩过的场景说起

你在《我的世界》里，对着草地按下右键，放下了一块石头

这一下点击，看起来只是「我屏幕上多了一个方块」。但如果这是在联机服务器上——

同一房间里还有 7 个人

他们在自己的电脑前，凭什么也能**同时**看到这块石头？

这块石头该不该被放下

如果那是别人的领地？如果你的背包里其实没有石头？**谁来判断**？

明天再上线，它还在吗

关掉游戏、重启服务器之后，凭什么这块石头**没有消失**？

- 你面前的画面，负责「看见」——但它不记住任何东西。游戏一关，它什么都不剩
- 远端的服务器，负责「判断」——它决定这件事能不能发生，也是唯一有资格做决定的一层
- 那本永不丢失的存档，负责「记住」——所有人看到的世界，最终都以它为准

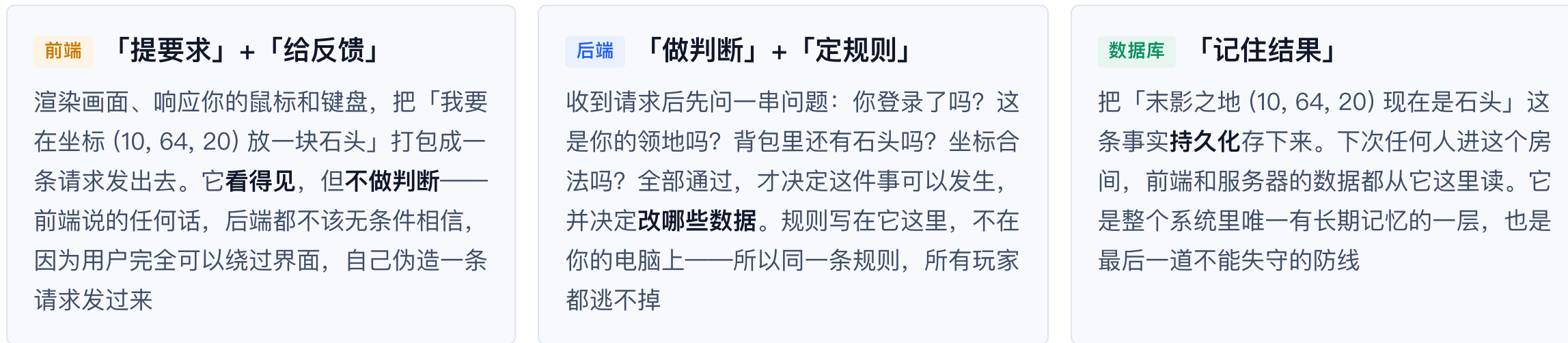
顺便把名字对上：你面前的画面 = **前端**（也叫客户端）；远端的服务器 = **后端**（也叫服务端）；那本存档 = **数据库**。以后你听到「客户端 / 服务端 / 存储」这些词，说的就是这三个角色

拆开看：一次「放方块」经过了几层

你按下右键的那 0.1 秒里，发生的事远比「屏幕上多了个方块」复杂



◀ ————— 结果原路返回，前端把方块画出来；后端同时把变化广播给同房间的其他玩家 ————— ▶



前端负责「我说的」，后端负责「算不算数」，数据库负责「记下来」 **后端是中间那个不能省的裁判**

换个场景：淘宝下单的那一秒

同样是「前端 → 后端 → 数据库」，换成钱的场景，每一层的重要性都变得更明显

- 1 你点了「提交订单」**
前端把你选的商品、收货地址、优惠券收集起来，打包成一条请求发给后端
- 2 后端先确认「你是谁」**
校验登录状态和身份凭证。没登录、凭证过期、身份对不上，一切免谈——这一步叫**鉴权**，是后端守的第一道门
- 3 再确认「这件事能不能做」**
库存够吗？商品下架了吗？地址在配送范围内吗？优惠券是不是你的、过期了吗？这些前端也判断了一遍，但**后端必须重做**——因为前端的结果可以被伪造
- 4 算钱、定秩序**
算总价、算运费、算优惠，并生成一个全平台唯一的订单号
- 5 落库：往数据库里写三件事**
写一条订单、扣减库存、记一条支付流水。**这三件事要么全成功、要么全不做**——这就是数据库「事务」存在的理由，也是后端最需要小心的地方
- 6 需要时，找别人帮忙**
调用支付、短信通知、物流系统等外部服务；注意：这些被调用的服务，本身也是别人写的后端
- 7 返回结果**
后端把「下单成功 + 订单号」返回给前端，前端跳转到支付页；你在手机上看到的，只有这最后一步

那能不能让前端直接连数据库？不行。那意味着每个人的数据库账号密码都要存在自己手机里，任何人都能查看和修改全平台的订单。所以数据必须由**后端代持**——这就是后端存在的第一个理由

你看到的只有「下单成功」四个字；后端在这背后，用几十到几百毫秒做完了一串判断和一堆数据操作

三个角色，各自管什么

把《我的世界》和淘宝这两个例子放在一起，抽象出来的就是这张表

对比维度	前端（看得见的那一层）	后端（做决定的那一层）	数据库（记得住的那一层）
核心职责	呈现与交互：把数据画出来，把你的操作收进去	判断与规则：校验、计算、编排，决定什么可以发生	存储与检索：把结果持久化，并快速查回来
最关心什么	好不好看、顺不顺手、手机上能不能用	对不对、快不快、稳不稳、安不安全	数据丢不丢、重不重、错不错、查得够不够快
典型技术	HTML / CSS / JavaScript、Vue、React	Java、Go、Mybatis、Spring Boot、SpringCloud、Gin、gorm、grpc	MySQL、Redis
出问题的表现	页面错位、按钮点了没反应	接口报错、响应变慢、金额算错、被人刷	数据查不出来、库存对不上、记录重复

后端的位置：它是唯一有资格「做决定」的那一层——既不相信前端传来的任何数据，也不让数据库裸奔在公网上；所有业务规则，最终都沉淀在这里

那后端工程师每天到底在写什么

如果你加入后端方向，两年后你日常打交道的就是下面这五件事

01 定义接口 (API)

说清楚「前端能调什么、要传哪些参数、会返回什么结果」。这是后端最日常的产出物——**接口就是后端和前端的合同**，合同定歪了，两边都要返工

02 写业务逻辑

把真实世界的规则翻译成代码：满减怎么算、限购怎么拦、活动时间怎么判定、异常情况怎么兜底

03 和数据库打交道

设计表结构、写 SQL、加索引；核心是两件事：**数据该长什么样，以及怎么查才快**；表设计错了，后面所有代码都会写得很别扭

04 保证不出乱子

一万个人同时抢 100 件库存，不能超卖；用户手抖点了两次支付，不能重复扣款；接口挂在公网上，不能被人刷、被人猜、被人注入；**不出事，比快更重要**

05 让系统扛得住

用缓存、异步、限流、扩容这些手段，让用户数从 100 涨到 100 万的时候，你的代码还跑得动。这一层决定你能负责多大的系统

你会发现，后端的工作大部分时间不是「敲代码」，而是「想清楚」——**然后用代码，把你想清楚的东西一行行落实**

第一行代码：Hello, World!

两门语言的第一课完全一样——让计算机对世界说一句你好

HelloWorld.java

javac → java HelloWorld

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

- **public class HelloWorld**: 定义一个类，类名必须和文件名一模一样
- **public static void main(String[] args)**: 程序的入口，计算机从这里开始执行
- **System.out.println(...)**: 把括号里的内容打印到屏幕上，然后换行

main.go

go run main.go

```
package main  
  
import "fmt"  
  
func main() {  
    fmt.Println("Hello, World!")  
}
```

- **package main**: 声明主包；Go 用包组织代码，只有 main 包能编译成可执行程序
- **import "fmt"**: 引入 fmt 包，它负责格式化输入输出，Println 就住在里面
- **func main()**: 程序的入口；注意 Go 没有类，直接写在函数里就行

Java 要求你先写一个类，Go 要求你先声明一个包——这是两门语言性格差异的第一处体现

学习路线：先建立认知，再按方向深入

前期跟着下面这条路线走，对后端有个基本认知、能自己做出点东西来；后期就按你自己的方向按需学习，灵活变化。

STAGE 1 打地基

基础知识 Git、MySQL、Redis、数据结构、计算机网络、计算机操作系统、计算机组成

STAGE 2 挑一条线，先做点东西出来

Java Java 语法 → MyBatis → Spring Boot → Spring Cloud

Go Go 语法 → GORM → Gin → gRPC

STAGE 3 按方向深入 到这一步就别再看路线图了，缺什么补什么

微服务与分布式

服务拆分、注册发现、分布式事务、消息队列、数据一致性

高并发与性能

缓存体系、限流降级、异步化、压测调优

中间件与云原生

Kubernetes、可观测性

Agent 开发

后端是 Agent 开发的基础

互联网这行业的学习和初高中的应试教育不同，没有固定的学习任务，没有固定的学习顺序，都要按照自己的期望的就业岗位去灵活学习

地基：七件绕不过去的事

这七样东西决定了你后面能走多远。但请看清每一样「什么时候开始学」——它们不该同时开工。

Git

代码的时光机，也是团队协作的通用语言。不会 Git，你连别人的代码都拿不到，更别说让别人 review 你的代码。

数据结构

数组、链表、哈希表、栈、队列、树、堆。它决定你写出来的代码是不是「不慢」，也是所有技术面试的第一关。

MySQL

关系型数据库，业务数据最终都要落在它这里。要会建表、会写 SQL，并理解索引为什么能让查询变快。

Redis

内存数据库。做缓存、计数器、排行榜、分布式锁，它是让系统从「能用」变成「快」的关键一环。

计算机网络

HTTP、TCP/IP、DNS。后端的本质就是「通过网络对外提供服务」，这一层不清楚，你写的接口就是个黑盒。

计算机操作系统

进程、线程、内存、IO、文件。程序真正跑起来的地方，也是性能问题最终的落脚点。

计算机组成

CPU、内存、磁盘、缓存层次。理解了性能为什么有上限，你做的优化才有依据，而不是凭感觉。

看完一门技术栈觉得基础不扎？那就去跟着教程去做项目，将技术栈都串联起来；项目来源：Github，B站，知识星球

Java 线和 Go 线：同一个骨架，两种语言

两条线的四站是对应的。你只需要走通其中一条，另一条以后一周就能捡起来。

● Java 线

01 Java 语法

变量与类型、流程控制、类与对象、集合框架、异常、泛型、IO、多线程

02 MyBatis

把 Java 对象和数据库表映射起来，用注解或 XML 写 SQL

03 Spring Boot

依赖注入、MVC 分层、参数校验、全局异常处理，写 REST 接口

04 Spring Cloud

注册发现、API 网关、配置中心、负载均衡、熔断降级

● Go 线

01 Go 语法

变量、切片与 map、结构体、接口、错误处理、goroutine 与 channel

02 GORM

把结构体映射成表，链式 API 完成增删改查与事务

03 Gin

路由、中间件、参数绑定与校验、统一响应封装

04 gRPC

用 Protobuf 定义接口，基于 HTTP/2 做服务之间的通信

两条线的四站是一一对应的：语法 → 连上数据库 → 对外提供服务 → 服务之间互通

学习的验收标准都不是「我看完了教程」，而是「我能拿它做出一个能跑起来的东西」

拥抱 AI，但别把脑子交出去

1 一定要用——AI 是这个时代最强的生产力工具之一

用它解释报错、生成样板代码、帮你读懂看不进去的源码、对比技术方案的取舍；拒绝工具的人，会被用工具的人替代，这件事在过去两年已经反复发生过

2 但决策权，必须留在你自己手里

AI 是执行者，你才是决策者；它能写出一段代码，但它判断不了「这个方案在业务上到底对不对」「这里的取舍是什么」「这个需求本身是不是合理」。方向错了，AI 只会更快地把你带到错的地方

3 因为在公司里，你要对你的代码负责

代码上线出了事故，承担后果的是写它的人，不是 AI，也不是工具；所以不管 AI 帮你写了多少，你至少要看得懂：看得懂每一行在干什么，说得清为什么这么写，改得动、调得出来、兜得住底

一条可执行的建议：每次 AI 给你一段代码，先别急着粘贴运行，问自己三个问题——① 它做了什么？能不能逐行讲清楚。② 为什么这么写？有没有更简单、更稳的写法。③ 如果它错了，我怎么发现？边界情况和异常路径想过了吗

一位面试官的回答是：答案便宜了，判断变贵了——AI 可以替你写代码，但替不了你判断



Show translation

面试快结束时，对方问了最后一个问题：你觉得 AI 时代最重要的能力是什么。

这种问题我一般都说点稳妥的。那天我说了实话。

我说判断力。

他让我展开。

我说我入行的时候，一个问题卡住三天是常事。要翻文档、要读源码、要在论坛上等人回。那三天里，答案是稀缺的，谁手上有答案，谁就有价值。

现在不是了。我三秒钟能拿到五个答案，每一个都工整、有条理、看起来都对。

难的不再是拿到答案，是从这五个里挑出对的那个，或者认出这五个全都不对。

这个能力没有捷径。它来自你自己踩过的坑、查过的日志、半夜回滚过的那些版本。

他问，那你怎么培养新人这个能力。

我说我也在想。以前新人是被 bug 教会的，现在 bug 还没出现，AI 就替他绕过去了。他省下了时间，也省掉了那次教训。

他没接话，在本子上写了一行。

出来的时候我想，其实这个问题我自己也没答案。

我只是比三年前更确定，答案便宜了，判断变贵了。

10:36 AM · Aug 28, 2026 · 4,044 Views

除了课程，你还可以从这四个地方学

课本和课堂告诉你原理，但它们不会告诉你行业里现在在用什么。下面这四个渠道会。

01 GitHub

全球最大的代码托管平台。去读别人的项目怎么组织目录、怎么写 README、怎么提交 commit；也要把自己的项目放上去——这是你未来简历上最有说服力的东西。不用怕做得差，第一个仓库本来就很粗糙。

02 B 站

免费的中文视频教程最集中的地方。入门阶段跟着一个完整的项目系列敲一遍，比自己啃文档快得多。但敲完必须能合上视频、自己复述一遍流程，否则等于没学。

03 技术博客

CSDN、稀土掘金，以及各大厂的微信公众号。遇到具体报错、具体配置问题时，博客往往是最快给出可执行答案的地方。但要有甄别：优先看带源码、带版本号和踩坑记录的文章，直接照抄容易踩到过期的坑。

04 与人交流

不只是实验室的学长学姐。多加几个技术交流群，认识一些同方向的朋友，把卡住的问题讲出来、听听别人怎么做，往往比一个人闷头查一天更有效。别怕问得简单——肯开口的人进步最快。

两个月，做出一个能跑起来的东西

不要求你现在就很强，但两个月后，我们希望看到你自己动手做出来的东西。

考核要求

- 期限：两个月，截止至 2026.11.13；提前完成的，也可以直接找我或者游学长
- 内容：用 Java 或 Go，做一个能跑起来的小型项目
- 答辩：向我阐述其中某些功能逻辑是如何实现的
- 技术栈：不限技术栈
- 是否能使用 AI：能，但你要理解 AI 写出来的逻辑

参考素材：可以跟着现成的项目做，但要理解

- B 站实战项目：黑马点评、苍穹外卖这类完整的项目，跟着敲一遍能很快建立起对后端全流程的认知
- GitHub 优秀开源项目：读真实的代码，从小小的、能跑起来的开始
- 知识星球（付费）：有质量较高的项目拆解和专栏，愿意花钱买时间的时候可以考虑
- 但记住：跟着敲完 ≠ 学会了。你必须能说清楚每一层在干什么

NOW IT'S YOUR TURN

打开你的 **IDEA / GoLand**, 写出你的第一个 **HelloWorld**

装好 JDK / Go SDK

→

新建一个项目

→

敲下那一行代码

→

运行

→

屏幕上出现 Hello, World!

你和后端之间的距离，就是这一行代码